Lecture 7 · Part II · Lecture notes

Neural networks

Dataset: Fashion-MNIST

How to use these notes

The primary text for this lecture is Géron, Chapter 9. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 The brief **2**

1.1 The corpus, and what it does not contain 2

2 Derivation: what a layer computes **3**

2.1 One layer, in matrices 3

3 From one unit to a network **4**

3.1 Price it before you fit it 4

3.2 The output layer, and the loss that must pair with it 4

3.3 How wide, and how deep? 5

4 Fitting it, and reading what happened **5**

5 Ten fits, and three things they do not tell you **6**

6 One number over ten classes **7**

7 Four walls, and they are all the same wall **8**

8 The notebook **8**

9 Design exercise: a radiology-software start-up's chest X-ray triage **8**

10 Exercises **11**

Summary **11**

---

1 The brief

A document-processing bureau scans incoming items and routes each to one of ten queues. Items arrive as small greyscale scans, already cropped and centred; a human currently sorts them, and the operator wants to know how often a machine would be right. Misrouting is recoverable but expensive — the item goes to the wrong desk and comes back a day later.

Ten classes, one label each, no ranking, no probability requirement stated. Write that down: it decides the output layer.

The last lecture had no labels at all: four hundred faces, compressed by PCA and clustered by k-means, with forty labels bought for the whole archive. Today every training image comes with its class.

The habit worth keeping

Not "accuracy is bad". *Count the classes before you choose.* Here every class holds exactly 6,000 images, so "always k" scores 10.0% and accuracy cannot hide a useless model.

1.1 The corpus, and what it does not contain

Fashion-MNIST: 60,000 training and 10,000 test images, 28×28 greyscale, ten balanced classes. We carve 5,000 validation images out of the training half with a fixed seed — 55,000 / 5,000 / 10,000 — because we are about to try ten configurations, and a number chosen on the set it is reported on is optimistic. The model is fitted on the first 12,000 of the 55,000, so the whole lecture runs on a free CPU, and every number below is from that run.

Failure condition — a pixel is a feature, and that costs something

We flatten each image into 784 numbers and hand it over as an ordinary table. The model is told nothing about which pixels are neighbours: shuffle all 784 columns with one fixed permutation and it learns exactly as well, though a human could not read a single shuffled image.
That is a real loss. Repairing it is four lectures away. Today we pay it, and it is the honest starting point.

Divide the pixels by 255, so every feature lies in $[0, 1]$. Networks are trained by gradient descent

with one step size shared by every weight, and inputs two orders of magnitude apart make that step too large for some weights and too small for others.

That division is not a leak. `StandardScaler` estimates a mean and a variance from data and must be fitted after the split; $x/255$ estimates nothing — it divides by a constant that is a property of the file format.

2 Derivation: what a layer computes

The vocabulary is historical. McCulloch and Pitts modelled a biological neuron that fires when enough of its inputs fire, which is where *neuron*, *activation* and *firing* come from. It is not an argument for anything; modern networks resemble brains about as closely as aeroplanes resemble birds.

A unit computes a weighted sum of its inputs plus a bias, then an *activation*: $h = \phi(\mathbf{w}^\mathsf{T}\mathbf{x} + b)$. The **multilayer perceptron** arranges units in layers: an input layer, which holds the features and has no weights; one or more hidden layers, each *fully connected* (dense) to the previous one, so that every unit receives every unit before it — which is where the parameters are; and an output layer, one unit per class. With two or more hidden layers the book calls it a *deep* network, and the word means nothing more than that.

2.1 One layer, in matrices

From one unit to a mini-batch

A single unit computes $z = \mathbf{w}^\mathsf{T}\mathbf{x} + b$: two numbers per unit and no more, a direction and how far along it to sit.
Stack n units side by side and the weights become a matrix, $\mathbf{z} = \mathbf{W}^\mathsf{T}\mathbf{x} + \mathbf{b}$ with one column per unit. The layer is *one matrix product*, which is why a tuned BLAS matters at all.
Feed a whole mini-batch by making $\mathbf{X}$ one row per instance: $\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{b}$, with shapes $(m \times d)(d \times n) \to (m \times n)$ and $\mathbf{b}$ broadcasting along the rows.
Apply a non-linearity elementwise:

$$\mathbf{H} = \phi(\mathbf{X}\mathbf{W} + \mathbf{b}).$$

Elementwise, so it changes no shape — the arithmetic above is untouched.

Now remove ϕ, and watch the stack collapse

Compose two layers with nothing between them:

$$(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 = \mathbf{X}(\mathbf{W}_1\mathbf{W}_2) + (\mathbf{b}_1\mathbf{W}_2 + \mathbf{b}_2).$$

Matrix product is associative, so the pair collapses to $\mathbf{X}\mathbf{W}' + \mathbf{b}'$ — one affine map. A stack of linear layers, however deep, is a linear model.

So ϕ is not a refinement

Without it there is no second layer at all. Two hundred stacked layers would still be a single affine map, and that is one line of algebra — it is the reason activation functions exist.

Activation	Range	Cost	Problem
Sigmoid	$(0, 1)$	an exponential	saturates at both ends
tanh	$(-1, 1)$	an exponential	saturates at both ends
ReLU	$[0, \infty)$	a comparison	flat below zero

$\max(0, z)$ is fast, does not saturate above, and in practice trains better. "Flat below zero" is a real defect with a name — dying ReLUs — diagnosed in Lecture 9, not here.

3 From one unit to a network

3.1 Price it before you fit it

```
784 x 300 + 300 = 235,500   layer 1
300 x 100 + 100 =  30,100   layer 2
100 x  10 +  10 =   1,010   output
                  -------
                  266,610   total
```

266,610 parameters fitted from 12,000 images — more than twenty parameters per training image. Lecture 3's failure condition for least squares was $m < n$, more parameters than instances. We are past it comfortably, and the model will still work. Understanding why is Lectures 13 and 14.

3.2 The output layer, and the loss that must pair with it

Ten mutually exclusive classes, so ten output units and a softmax across them:

$$\hat{p}_k = \frac{\exp(z_k)}{\sum_{j=1}^{10} \exp(z_j)}.$$

Every output is positive and they sum to 1, and the largest logit gives the largest probability, so `argmax` is unchanged by the softmax. The loss is cross-entropy,

$$L = -\sum_{k=1}^{10} y_k \log \hat{p}_k = -\log \hat{p}_{\text{true}},$$

since the target is one-hot and nine of the ten terms are zero. Chapter 4 called this log loss when there were two classes; it is the same object.

Chapter 10 summarises the choices for classification networks in one table:

Task	Output units	Output activation	Loss
Binary	1	sigmoid	binary cross-entropy
Multilabel binary	one per label	sigmoid	binary cross-entropy
Multiclass	one per class	softmax	cross-entropy

Ours is the third row. The distinction between rows two and three is not cosmetic: multilabel asks ten independent yes/no questions, multiclass asks one ten-way question, and softmax couples the outputs where ten sigmoids do not.

Failure condition — the pairing is not optional

Softmax with mean squared error trains, slowly and badly: the gradient through a saturated softmax nearly vanishes, and MSE does nothing to undo it. Cross-entropy is the loss that cancels the softmax derivative, which Lecture 17 derives. Until then, take the pairing from the table.

Two names arrive here and are cashed later. **Backpropagation** is one forward pass to compute the loss, one backward pass to compute every partial derivative, and one descent step — and the backward pass costs about as much as the forward one *no matter how many parameters there are*. 266,610 parameters, one number of interest, and a single sweep gets all the derivatives; that claim is derived at the top of the next lecture and is the whole reason any of this is feasible.

3.3 How wide, and how deep?

Depth buys parameter efficiency: two narrow layers express things one wide layer needs exponentially many units for. Width buys capacity within a layer, at $d \times n + n$ parameters a time. In practice, pick a depth that is enough, then grow the width until validation stops improving — and stop there.

There is no formula. What there is, is a budget you can compute before fitting anything — the parameter count — and a validation curve that tells you when you have spent it badly. Not examinable — engineering: the numbers depend on the corpus; the method for finding them does not.

4 Fitting it, and reading what happened

Quantity	Measured
Training images	12,000
Epochs	20
Validation accuracy	86.74%
Test accuracy	85.63%
Baseline	10.0%

Hold on to this model — the next lecture writes the same one again, in PyTorch.

The model is one `MLPClassifier(hidden_layer_sizes=(300, 100), activation="relu", solver="adam", learning_rate_init=1e-3, batch_size=128, max_iter=20, random_state=42)`, and every architectural decision above is one of its arguments. The softmax and the cross-entropy are not arguments: the class chooses them for you, from the number of classes it sees in the labels.

The learning curves separate and stay separated, and that gap is overfitting — the training-versus-validation gap, drawn as lines. The loss falls smoothly, so the optimiser is working; training accuracy keeps climbing, so the model has capacity to spare; validation accuracy flattens, so the extra capacity is being spent on memorising. And the curve answers a question the number

cannot: would more epochs help? Hardly — over the last ten epochs validation moved less than a point while training gained three, and without the curve you would have guessed.

5 Ten fits, and three things they do not tell you

Five architectures and five learning rates, on a 6,000-image subset for 8 epochs so they fit in the lecture.

Hidden layers	Parameters	Validation accuracy
30	23,860	82.68%
100	79,510	83.80%
300	238,510	83.88%
300, 100	266,610	84.10%
300, 200, 100	316,810	85.12%

Learning rate	Validation accuracy
0.0001	81.58%
0.0003	83.46%
0.001 — the default	84.10%
0.003	84.36%
0.01	83.40%

On this subset *depth pays, a little*: a second hidden layer adds about a fifth of a point and a third about a point more, for a fifth more parameters. And the library's default learning rate is not the best value on the grid.

Failure condition — a sweep read as a law

The five architectures span 2.4 accuracy points; the five learning rates span 2.8. On this grid one badly chosen scalar loses more than any structural choice gains — which is a claim about a grid and a scale, never about the world, and why a sweep must always be reported with the size it was run at.

Ten fits is also not a search: ten points from a space with five axes, varied one axis at a time with everything else held fixed. You have not optimised anything; you have looked at ten places.

Does the ranking survive more data?

Hidden layers	6,000 images, 8 epochs	12,000 images, 20 epochs
30	82.68%	85.38%
100	83.80%	86.26%
300	83.88%	86.58%
300, 100	84.10%	86.74%
300, 200, 100	85.12%	87.14%

Here it does: the same order at both scales, and each architecture gains two to three points from the larger run — as much as the whole spread between the five on either. A ranking measured on a fraction of your data is a measurement of that fraction: a deeper network needs more data to pay for its extra parameters, so on another corpus, or a smaller fraction, the order need not hold. The only way to know is to run the comparison again on all the data.

6 One number over ten classes

Split it. Lecture 3 built the confusion matrix for two classes and the object generalises without change: row i, column j counts the images of class i the model called class j.

Recall on *Pullover* is 63.6%, against 97.2% on *Trouser*. Pullovers are called Coat 18.8% of the time, Shirt 12.7%, and Dress 2.0% — Coat and Shirt, two of the classes anyone would name from the image grid, take 31.5 of the 36.4 points it loses, and Shirt is the next-worst class at 71.1%. *The model is not confused about anything a person would find easy.*

A defensible report

"Overall it is right 85.6% of the time. But it is right 63.6% of the time on Pullover, and almost all of those errors go to two visually similar queues. If those three desks are adjacent, the practical error rate is much lower than the headline. If they are in different buildings, it is much worse."
The headline accuracy does not answer the operator's question. The per-class breakdown does, and it took one extra function call.

7 Four walls, and they are all the same wall

What you want	What happens
Make it faster	there is no `device=`. Scikit-Learn does not support GPUs and says so in its own FAQ — a deliberate scope decision, not an oversight. PyTorch, from the next lecture, gives you the choice, and Lecture 8 measures when it is worth taking
Change the loss	`loss="mae"` raises `TypeError`. Cross-entropy is hard-coded
See a gradient	`[a for a in dir(clf) if "grad" in a.lower()]` is empty. 266,610 partial derivatives were computed on every batch and none was kept
Stop part-way	`partial_fit` is the finest handle there is, and one call is one complete pass over everything you hand it

Each of these matters for an ordinary reason, not an exotic one. A weighted loss is one line in a framework that lets you write one, and the operator saying "calling a coat a shirt is cheap, calling a bag a sandal is expensive" is the normal case. Lecture 9 builds a twenty-layer network whose loss barely moves, and the diagnosis is a per-layer gradient statistic you cannot compute here.

Failure condition — the problem underneath all four

`fit()` contains a `for` loop over epochs and batches that you did not write, cannot see, and cannot enter. Every one of the four walls is that one wall.
This is not a criticism of Scikit-Learn — `MLPClassifier` is documented as a convenience, and for a tabular problem with a standard loss it is the right tool. It is the wrong tool from here on, and now you know precisely why.

8 The notebook

What to change

Remove the activation — set it to identity — and refit. Accuracy collapses towards what a linear model gets, which is the collapse derived in §2, measured.

9 Design exercise: a radiology-software start-up's chest X-ray triage

The lecture closes with a design exercise in the manner of an ML system-design interview, on the system behind a company's product, with the shape of data this model suits — and, this time, a medical device.

The task. A radiology-software start-up sells a plug-in for the hospital worklist: for every chest X-ray that arrives it raises a flag — *may show a tumour* — so that a radiologist reads that scan first. It reorders the queue; it never diagnoses. Hospitals pay for a shorter wait from scan to read on the tumours; a false flag costs radiologist minutes; a miss reaches a patient. And it is a medical device: cleared for one stated intended use, and every change to the model is a change the regulator must see. Given: an archive of past scans from partner hospitals, each with

the radiologist's report and, where there was one, the biopsy result, grouped by patient and by hospital. Hundreds of thousands of scans, tumours in a few per hundred; thousands of scans a day across the customers, each flagged within seconds. The output feeds the worklist, the radiologist, the hospital's quality team and the regulator's file. The model at the centre is today's: a multilayer perceptron on the downsampled pixels, fitted with `MLPClassifier`, the first model and the baseline the next must beat.

1. **Frame it honestly: the intended use decides the metric.** One row per scan, two exclusive classes — a tumour in the report, or not — written down first, because it decides the output layer. Count the classes first: tumours are a few per hundred, so "never flag" scores far above ninety and accuracy hides a useless model; the metric is recall on the tumour class at the share of scans the worklist may flag, with the confusion matrix beside it, and the two trivial baselines are never flag (high accuracy, no recall) and flag everything (full recall, nothing reordered). *Pitfall:* the pitch says "right on more than nine scans in ten", which never flagging also achieves. The regulator asks for sensitivity and specificity at the operating point, under an intended use written in one sentence — a triage aid that reorders a queue, not a diagnosis. Written first, it decides the metric now and the device's label later.

2. **A pixel is a feature, and the split is by patient.** Downsample every scan to one small fixed size and flatten it into one column per pixel; the model is told nothing about which pixels are neighbours, a real loss paid today, and dearest on a small nodule. Divide by the largest pixel value, a property of the file format that fits nothing, so it is not a leak; carve a validation set from the training half with a fixed seed and read the test set once. *Pitfall:* a random split over scans puts one patient's follow-up — the same lungs, the same tumour a month later — on both sides, and the model is scored on tumours it has seen. Split by patient, and hold out a whole hospital as the test set: the device will be sold to hospitals and scanners that are not in the archive, and that is the number the regulator's file must show.

3. **Count the parameters before you fit.** A dense layer from d inputs to n units costs $d \times n + n$ parameters, and the first layer holds almost all of them; that count against the number of training scans is the budget — pick a depth that is enough, grow the width until validation stops improving, and stop there. *Pitfall:* the scanner delivers millions of pixels a scan, the engineer flattens them as they come, and the first layer alone is billions of parameters for a few hundred thousand scans, so nothing fits in memory and nothing would generalise. Downsample to the size the count was done for, agreed with the radiologists so that the nodule survives it; write the arithmetic down and check it against what the fitted model reports.

4. **The output layer and the loss, paired from the table.** Tumour or not is the binary row: one unit, a sigmoid, and binary cross-entropy — minus the log probability of the right answer. `MLPClassifier` chooses both from the labels. *Pitfall:* an engineer who wants "normal, benign, tumour" for the radiologist builds three sigmoids, independent yes/no questions whose answers need not sum to one, so the threshold the device is cleared at has no meaning; softmax with squared error trains slowly and badly, and the pairing that would have worked is never suspected. Take the row from the table, and add classes only when the intended use says so.

5. **Learning curves, and a sweep on a subset.** Record the loss and the validation recall at every epoch: training still climbing while validation flattens says the extra capacity is memorising scans. Stop where validation stops moving and read the held-out hospital once; tune width,

depth and the learning rate by hand on a subset, one axis at a time, then rerun the best few on all the scans, and log every fit, because the regulator asks how the model was chosen. *Pitfall:* ten fits is not a search, and a ranking measured on a fraction of the archive is a measurement of that fraction — the deeper network that lost on the subset may win on all the scans, and one badly chosen learning rate lost more on today's grid than any structural choice gained.

6. **The confusion that matters clinically.** One number hides the shape: build the confusion matrix and read its two errors apart. A miss leaves a tumour waiting in the queue like any other scan; a false flag costs a radiologist minutes on a normal one; they are not the same price. Then recall per subgroup — small nodules against large, each hospital's scanner, portable against fixed — so that the misses have an address. *Pitfall:* the headline recall is defended at the hospital's quality meeting while the misses sit in one place, the small nodules the downsampling erased or the one hospital with an older scanner, and those are the ones that reach a patient. Report recall per subgroup, which errors go where, and the price of each; the regulator's file carries the per-hospital table, not the headline.

7. **Abstain to the radiologist, and never say "clear".** The sigmoid output is the model's score; sweep the threshold on the validation set and draw two curves, the share of scans flagged and the recall among the tumours, and let the radiologists pick the point, because their minutes and a miss both have prices. Three outputs reach the worklist: flagged, not flagged, and not assessed — for a scan whose metadata or reconstruction error says it is unlike the archive, a lateral view or a child's chest. *Pitfall:* the sigmoid must give a number for every scan, so a lateral view or a pacemaker gets a confident score of something; and a radiologist who reads "not flagged" as "normal" reads less carefully — the device has started to diagnose. Bucket the validation scans by score and check that flagged scans are tumours as often as the number claims; show the score only if that check passes, and abstain, visibly, on anything out of scope.

8. **In the product: what `MLPClassifier` cannot do.** The hospital will ask for a loss that charges a miss far more than a false flag, for a flag within seconds at the department's volume, for a per-batch log in the device file and a stop when a batch looks wrong, and for a model that knows which pixels are neighbours, for the small nodule — and the wrapper has cross-entropy hard-coded, one CPU, one epoch as its smallest unit of control, and a table of pixels; the convolution comes later. *Pitfall:* the prototype ships as the device: the weighted loss has no argument to go in, and a quiet retrain for better recall puts a model in hospitals that the regulator never saw. All four walls are one wall — `fit()` holds a loop you did not write. Keep the wrapper for the prototype and its baseline; the product's model runs in a framework whose loop you own, every retrain is a numbered version in the device file, released hospital by hospital in shadow mode — flags computed, not shown, compared with the radiologists' reads — then monitored on the share flagged and on the misses the quality team finds, with a clinical safety officer on call.

9. **Ship it.** Archive → downsampled, scaled, one row per scan → split by patient, a hospital held out → the parameters counted → one unit, sigmoid, binary cross-entropy → curves, stop where validation flattens, the subset ranking rerun → confusion matrix and recall per subgroup → a threshold chosen on validation, abstention on the out-of-scope → the held-out hospital once → a numbered version, shadow mode, monitored.

10 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 8's deck and in the exercise book.

1. A multilayer perceptron with the activation removed is written as $W_3W_2W_1x$. State what function class it can represent, and why depth then buys nothing. [4]
2. Give the parameter count of a dense layer from m inputs to n units, and evaluate it for the first layer of a network on 28×28 images with 300 units. [4]
3. A ten-class classifier for mutually exclusive classes is built with ten sigmoid output units and binary cross-entropy. Name the row of the classification architecture table this is, and the row it should be. [4]
4. Fashion-MNIST is exactly balanced across ten classes. State the trivial baseline's accuracy, and why balance is what makes accuracy defensible here. [3]
5. An architecture sweep finds the best hidden-layer size on this dataset. State what that result does and does not transfer to. [4]

Summary

A layer is one matrix product and one elementwise non-linearity, $\mathbf{H} = \phi(\mathbf{XW} + \mathbf{b})$, and without ϕ any stack collapses to a single affine map by associativity — so depth exists only because the non-linearity does. The parameter count $d \times n + n$ per layer lets you price a network before fitting it: 266,610 for ours, from 12,000 images.

The classification table answers three forced questions — how many output units, which activation, which loss — and the loss must pair with the activation rather than being chosen beside it. With the classes counted first, accuracy is defensible, and the network reached 85.63% test accuracy against a 10.0% baseline on one CPU.

Ten hand fits gave three lessons, none of them about the winner. Depth paid a little on 6,000 images; the learning rate mattered slightly more than the architecture *on that grid at that scale*; and the ranking had to be checked on all 12,000 images before it could be trusted — here it held, which is a measurement, not a rule.

Then four things `MLPClassifier` will not do — run on a GPU, change the loss, expose a gradient, or stop inside an epoch — which are all the same thing: the training loop was written for you, and you cannot enter it.

The closing design exercise built a radiology-software start-up's chest X-ray triage in nine steps — recall at the operating point under a written intended use, a split by patient with a hospital held out, the parameters counted at the downsampled size, the binary row of the table, the curves read for the stopping epoch, the confusion that matters clinically, abstention to the radiologist, and the loop owned in a numbered version of a medical device run in shadow mode — each step named for the pitfall it avoids.